Environment Setup Guide, Part 3

Native Ubuntu migration, the project image, and running the full testbed
SDN-Orchestrated Network Slicing for Vehicular 5G (V2X). November 2026.

Parts 1 and 2 set the testbed up inside a VirtualBox VM on Windows. This part covers the move to a dedicated Ubuntu laptop, rebuilding the images, and the scripts that now run every check and experiment. It continues the step numbering from Part 2.

Step 14: Host machine

Item	Value
Operating system	Ubuntu 24.04.5 LTS, X11 session
Kernel	6.8.0 generic (the handoff recorded 6.8.0-142; the laptop later ran 6.8.0-147)
Hardware	12 cores, 15 GB RAM
Docker	29.1.3 from the Ubuntu package (`docker.io`)
Kernel modules	`mac80211_hwsim` and `openvswitch` must load. Check with `lsmod | grep -E 'hwsim|openvswitch'`
Host Open vSwitch	Must stay **inactive**. OVS runs inside the container, and a host OVS service would fight over the kernel datapath
Other containers	Stop unrelated containers during experiments so they don't take CPU

The user is not in the `docker` group, so every Docker command needs `sudo`. That is why each script below asks for the password once at the start.

Step 15: Base image `mn-wifi:v1`

Clone Mininet-WiFi and patch its Dockerfile before building. The base image is `python:3.9`, now Debian 13. Insert these two lines before `RUN util/install.sh -Wlnfv6`:

```
RUN sed -i "s/from FlightRadarAPI import FlightRadar24API/from FlightRadar24 import
FlightRadar24API/" mn_wifi/net.py

RUN sed -i "s/matplotlib.use('TkAgg')/matplotlib.use('TkAgg', force=False)/"
mn_wifi/telemetry.py
```

- **FlightRadar fix.** FlightRadarAPI 1.4.1 and later need Python 3.10. On 3.9, pip installs 1.4.0, whose module is named `FlightRadar24`.
- **TkAgg fix.** Upstream now forces the Tk plotting backend at import, which crashes the headless Docker build.
- The `set_data([x],[y])` patches from Part 1 Step 7.3 are no longer needed; upstream fixed them.

```
git clone https://github.com/intrig-unicamp/mininet-wifi ~/mininet-wifi

cd ~/mininet-wifi && sudo docker build -t mn-wifi:v1 .
```

Step 16: Project image `mn-wifi:v2`

Built from `~/v2x-project/docker/Dockerfile`. It adds RYU, restores Mininet-WiFi's `mn`, and installs `iperf3`:

```
FROM mn-wifi:v1

RUN python3 -m pip install "setuptools<68" wheel \

 && python3 -m pip install --no-build-isolation ryu \

 && python3 -m pip install "eventlet==0.30.2"

RUN install -m 755 /app/mininet-wifi/bin/mn /usr/local/bin/mn

RUN apt-get update && apt-get install -y iperf3 && rm -rf /var/lib/apt/lists/*

WORKDIR /app/project
```

- **RYU 4.34 pins.** `setuptools<68` because `get_script_args` was removed; `--no-build-isolation` so that pin applies; eventlet 0.30.2 because `ALREADY_HANDLED` was removed later.
- **`mn` restore.** Plain Mininet's `mn` overwrote Mininet-WiFi's, so `mn --wifi` failed until it was reinstalled.

```
cd ~/v2x-project/docker && sudo docker build -t mn-wifi:v2 .
```

Step 17: Running the testbed

17.1 Manual session (two terminals)

The original launch script, `~/start_mnwifi.sh`, starts a privileged container with the project folder mounted at `/app/project`. Everything else in the container is lost on exit.

```
~/start_mnwifi.sh

mn -c

ryu-manager --wsapi-port 8081 /app/project/controller/v2x_orchestrator.py
```

In a second terminal:

```
sudo docker exec -it $(sudo docker ps -q --filter ancestor=mn-wifi:v2) bash

python3 /app/project/v2x_topology_v2.py
```

Always run `mn -c` before starting RYU, because `mn -c` kills `ryu-manager`. Wait about 25 seconds for the scripted mobility to finish before measuring.

17.2 Scripted runs (one password each)

Command	What it does
`sudo bash scripts/run_all_host.sh 5`	Everything: functional checks, Step 12 check, a gate that stops on failure, dashboard recordings, then 5 experiment repeats. About 3 hours. Progress is in `logs/final/stage` and `logs/final/batch.log`
`sudo bash scripts/verify_step12_host.sh`	Step 12 verification only, with the live plot (run `DISPLAY=:1 xhost +local:docker` first)
`sudo bash scripts/run_step2_host.sh 1 "heavy"`	Selected experiment scenarios only
`sudo bash scripts/run_live_demo_host.sh orchestrated 30`	A continuous testbed for the dashboard's Live view, for 30 minutes

All of these run from `~/v2x-project`, keep their logs under `logs/`, and hand the files back to your user when they finish.

17.3 Dashboard

```
python3 ~/v2x-project/dashboard/server.py
```

Then open `http://127.0.0.1:8090`. It needs no sudo, no Docker and no internet, and replays the recordings in `dashboard/recordings`.

Step 18: Problems found on the native setup, and their fixes

Symptom	Cause	Fix
No switches register with RYU; hundreds of dead connections on port 6653	The orchestrator's `self.events` overwrote RYU's own event queue	Renamed to `self.event_log`
Vehicle-to-vehicle pings fail across RSUs after the vehicles move (74/90)	The old RSU keeps a departed vehicle in its station list for minutes and delivers its frames locally	hostapd settings from Saraiva et al.: `ap_max_inactivity=6`, `skip_inactivity_poll=1`, `max_listen_interval=6`, plus 1-second vehicle beacons
Traffic between two vehicles on one RSU bypasses its slice meter	The RSU's kernel bridges the frames itself, so they never reach OVS	`client_isolation=True`, and the controller sends that traffic back out the Wi-Fi port (`OFPP_IN_PORT`)
Traffic gaps while vehicles move	Partly a coverage hole between the RSUs; the rest (7 to 14 s per handover) is the emulated station re-associating between RSUs on channels 1 and 6	`range=30` on the RSUs removes the hole ("effective 30.00m using tx power 13 dBm"). The re-association delay remains and is documented as a limitation
Uplink traffic gets neither the 5 ms delay nor the 20 Mb/s limit	The RSU end of each backhaul link had lost its TCLink shaping (no root qdisc at all)	The topology rebuilds both ends: HTB at 20 Mb/s, then 3-band priority, then 5 ms netem per band
`sch_htb: quantum ... is big` warnings	TCLink's HTB class at 20 and 100 Mb/s	Harmless. The rebuilt backhaul classes set `quantum 1514`, but TCLink still prints the warning when it first creates the links
`addLink` returns `None`	Mininet-WiFi only creates links during `build()`	Find the backhaul links in `net.links` after building

Step 19: Verification status

Check	Status	Evidence
Images build and `mn --wifi` works	Done	Handoff notes, 5 November
Orchestrator connects to all 3 switches	Done	`logs/step12`
Full connectivity after mobility	Done	90/90 in all three pingalls, `logs/step12`
Meters limit URLLC, mMTC, best effort and eMBB	Done	`logs/final/functional`
REST resize, including bad requests	Done	200, 200, 404, 400 in `logs/final/functional`
Priority queue tree and DSCP filters installed	Done	`logs/final/functional/qdisc_tree.txt`; band 1 carries the marked traffic (`qdisc.txt` in each trial)

Full experiment, 5 repeats (80 trials)	Done	`logs/step2/summary.md`, figures in `docs/figures`